# 山东大学 计算机科学与技术 学院

# 数据结构与算法 课程实验报告

| 学号：202500130002 | 姓名： 林毅玮 | 班级： 25.2 |
|---|---|---|
| 实验题目：简单排序算法 | | |
| 实验学时： | 实验日期： 2026/9/18 | |
| 实验目的：<br>练习排序算法 | | |
| 软件开发环境：<br>VS code | | |

## 1. 实验内容

要求：在不使用任何与实验相关的 STL 的前提下，使用 C++ 类模板（template <class T>）定义一个泛型的排序类。

数据管理：利用动态内存分配创建动态数组来存储排序数据。程序支持由键盘输入数据规模 $n$，并由系统自动随机生成 $n$ 个在 $0 \sim 1000$ 之间的整数初始化数组。为了保证对比的公平性，类中维护了一个原始数据的备份，确保每次排序算法面对的初始乱序序列完全一致。

算法实现：
实现四种特定的排序算法：名次排序、及时终止的选择排序、及时终止的冒泡排序、插入排序。
在算法执行过程中，输出排序的中间状态（如每趟循环后的数组元素排列情况）。
设计计数器，精确统计并输出每种算法在同一组数据上所耗费的比较次数与数据移动或者赋值次数。

## 2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

数据结构：
本实验采用封装了动态一位数组的模板类 Sorter<T> 作为核心数据结构。
通过指针 T* data 和 T* originalData 指向在堆区动态分配的连续内存块。data 用于执行排序操作，originalData 用于保存原始乱序状态以供重置。
数据类型是利用模板参数 T，该数据结构可以无缝切换为存储 int、float 或 double

算法描述：

名次排序：
算法分配一个与原数组等长且初始值为 0 的辅助数组 rank。通过双重循环，将每个元素与其余元素进行比较。若 data[j] <= data[i]，则 data[i] 的名次 rank[i] 加 1。统计完毕后，根据 rank 数组的指引，将原数据放入临时数组 temp 对应的位置，最后拷回原数组。

及时终止的选择排序：

在外层循环确定当前位置 $i$ 的最小元素时，内层循环不仅负责寻找最小值索引 minIndex，还同时检测剩余未排序序列是否满足 data[j - 1] <= data[j]。如果一趟扫描下来，不仅当前位置无需交换（minIndex == i），且剩余序列已经完全有序，则触发 break 提前结束整个排序。

及时终止的冒泡排序：
在外层循环开始前设置布尔变量 swapped = false。在内层相邻元素两两比较的过程中，若发生前大后小的情况则进行交换，并置 swapped = true。如果内层循环结束后 swapped 依然为 false，说明本趟遍历未发生任何交换，序列已完全升序，直接提前终止外层循环。

插入排序
将数组首元素视为已排序序列。从第二个元素开始，将其暂存到变量 key，并在已排序序列中从后向前扫描。若发现大于 key 的元素，则将其向后移动一位。直到找到小于或等于 key 的元素，将 key 插入到该元素的后方

## 3. 测试结果（测试输入，测试输出）

按照题目给的测试一样

## 4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

时间空间复杂性探讨：

名次排序比较次数始终是严格的 $\frac{n(n-1)}{2}$,时间复杂度为稳定恒定的 $O(N^2)$。移动次数非常少,固定为 $2N$。

但其代价是需要分配额外的 rank 数组和 temp 数组，空间复杂度为 $O(N)$。它非常适合“比较开销小，但元素移动开销极大”的场景。

插入排序：空间复杂度为 $O(1)$。在本次测试中表现优秀，因其只在必要时才移动元素。它的平均时间复杂度为 $O(N^2)$，但常数项极小。

冒泡排序的优化：传统的冒泡排序面对 5 个元素固定需要比较 10 次。在本次测试中，由于第三趟扫描后检测到数组已有序（未发生交换），比较次数降为了 9 次并提前结束。当输入数据处于“基本有序”状态时，该机制能将其时间复杂度从 $O(N^2)$ 直接降级至 $O(N)$，优化效果极佳。

选择排序的优化：传统选择排序在寻找最小值时必须遍历到底。引入提前终止后，程序不仅找最小值，还顺带做了“有序性”体检。如果数组后半段天然有序，它也能省去后续无意义的外层遍历。不过，这一优化增加了内层循环的判断分支（增加了比较次数），在极端乱序的情况下，可能会导致常数级开销略微上升。

在本程序中，插入排序和冒泡排序由于只在严格大于 (>) 时才交换/后移，天然保证了稳定性
。
名次排序在代码中通过 data[j] <= data[i] 的判断，使得相等的元素中，排在后面的元素名次会更大，从而巧妙地避免了位置冲突并保证了稳定性。

选择排序是不稳定的（例如序列 $5,5,2$，第一趟交换会将第一个 5 换到末尾），这是该算法先天的性质决定的。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include <iostream>

template <class T>
class Sorter {
private:
    T* data;
    int length;

public:
    //使用动态函数动态分配内存
    Sorter(int n) {
        length = n;
        data = new T[length];
    }

    //使用析构函数释放内存
    ~Sorter() {
        delete[] data;
    }

    //输入数据
    void input() {
        for (int i = 0; i < length; i++) {
            std::cin >> data[i];
        }
    }

    void print() {
        for (int i = 0; i < length; i++) {
            std::cout << data[i] << " ";
        }
        std::cout << std::endl;
    }

    //名次排序
    void rankSort() {
        int* rank = new int[length]();

        for (int i = 1; i < length; i++) {
            for (int j = 0; j < i; j++) {
                if (data[j] <= data[i]) {
                    rank[i]++;
                } else {
                    rank[j]++;
```

```
                }
            }
        }

        //根据名次重新排列成临时数组
        T* temp = new T[length];
        for (int i = 0; i < length; i++) {
            temp[rank[i]] = data[i];
        }

        for (int i = 0; i < length; i++) {
            data[i] = temp[i];
        }

        delete[] rank;
        delete[] temp;
    }

    // 及时终止的选择排序
    void selectionSort() {
        for (int i = 0; i < length - 1; ++i) {
            int minIndex = i;
            bool isSorted = true; //用来检测剩余的部分数据是否已经有序

            for (int j = i + 1; j < length; j++) {
                if (data[j] < data[minIndex]) {
                    minIndex = j;
                }
                //检查是否已经完全升序
                if (data[j - 1] > data[j]) {
                    isSorted = false;
                }
            }

            // 如果剩余的序列已经有序并且当前就是最小值，那么提前终止
            if (isSorted && minIndex ==  i) {
                break;
            }

            if (minIndex != i) {
                T temp = data[i];
                data[i] = data[minIndex];
                data[minIndex] = temp;
            }
        }
    }
```

```
    //及时终止的冒泡排序
    void bubbleSort() {
        for (int i = 0; i < length - 1; i++) {
            bool swapped = false;
            for (int j = 0; j < length - 1 - i; j++) {
                if (data[j] > data[j + 1]) {
                    T temp = data[j];
                    data[j] = data[j + 1];
                    data[j + 1] = temp;
                    swapped = true;
                }
            }

            //如果有一趟冒泡排序没有发生交换，说明这一轮已经有序，终止这一趟
            if (!swapped) {
                break;
            }
        }
    }

    // 插入排序
    void insertionSort() {
        for (int i = 1; i < length; ++i) {
            T key = data[i];
            int j = i - 1;
            while (j >= 0 && data[j] > key) {
                data[j + 1] = data[j];
                j = j - 1;
            }
            data[j + 1] = key;
        }
    }
};


int main() {
    int n;
    std::cin >> n;

    // 进行模版的实例化
    Sorter<int> sorter(n);

    //读取数据
    sorter.input();
```

```
    //计数排序
    sorter.rankSort();

    //及时终止的选择排序
    sorter.selectionSort();

    //及时终止的冒泡排序
    sorter.bubbleSort();

    //插入排序
    sorter.insertionSort();

    //打印结果
    sorter.print();

    return 0;
}
```